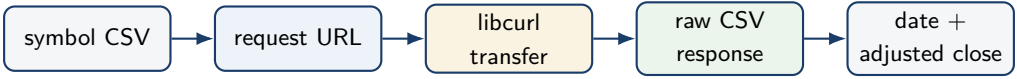
Historical and Market Data with libcurl

FRE6883 – Recitation 13

Request, receive, parse, validate, clean up

From a ticker file to usable prices



Session goal

Trace every ownership boundary in the pipeline, complete the key callbacks and parsing code, and diagnose failures without relying on a live API during class.

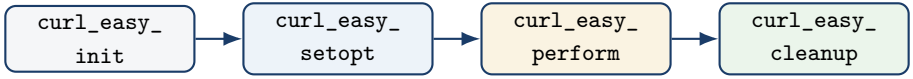
What this recitation emphasizes

- the `curl_easy` handle lifecycle;
- callback functions and the data pointer passed to them;
- file input, CSV parsing, and string conversion;
- exceptions and cleanup across failure paths;
- separating network transfer from data interpretation.

Scope

We use the interfaces and coding style from Topic 11. Gnuplot belongs to Topic 12, so visualization is only a transition at the end.

The libcurl easy-interface lifecycle



Mental model

Create one handle, configure what and where to transfer, perform the synchronous request, then release the handle.

01 | Put the calls in order – Problem

GUIDED CODE Handle lifecycle

Reorder before revealing

```
curl_easy_cleanup(handle);
CURLcode status = curl_easy_perform(handle);
CURL* handle = curl_easy_init();
curl_easy_setopt(handle, CURLOPT_URL, url.c_str());
```

Which call can fail by returning a null pointer? Which call returns the transfer status?

01 | Put the calls in order – Solution

```
CURL* handle = curl_easy_init();
if (!handle)
    throw std::runtime_error("Curl init failed");

curl_easy_setopt(handle, CURLOPT_URL, url.c_str());
CURLcode status = curl_easy_perform(handle);
curl_easy_cleanup(handle);
```

Two kinds of status

Initialization is checked through the returned pointer. The completed transfer is checked through the `CURLcode`.

The write callback answers one question

When libcurl receives a block of bytes, it calls a function shaped like

```
callback(ptr, size, nmemb, destination).
```

The block contains

```
size * nmemb
```

bytes. The callback stores those bytes and returns the number successfully handled.

Important

The fourth argument is the pointer previously supplied through `CURLOPT_WRITEDATA`.

One transfer, two possible destinations

File destination

The destination points to a `FILE*`. The callback forwards bytes to `fwrite`.

Best for inspecting the raw response.

Memory destination

The destination points to a `MemoryStruct`. The callback appends bytes for parsing.

Avoids repeated file operations.

The lecture's in-memory buffer

```
struct MemoryStruct {
    char* memory;
    size_t size;

    MemoryStruct() : memory(NULL), size(0) {}
};
```

Invariant

After a successful append, `size` is the number of response bytes and `memory[size]` is the null terminator.

02 | Complete the file callback – Problem

GUIDED CODE Callback contract

Return the amount handled

```
int write_data(void* ptr, int dataSize,
               int nmemb, FILE* outputStream)
{
    size_t written = /* write the block */;
    return /* ? */;
}
```

If `dataSize=1` and `nmemb=512`, how many bytes should a successful callback report?

02 | Complete the file callback – Solution

```
int write_data(void* ptr, int dataSize,
               int nmemb, FILE* outputStream)
{
    size_t written =
        fwrite(ptr, dataSize, nmemb, outputStream);
    return (int)written;
}
```

Course-code convention

With `dataSize=1`, `fwrite` returns 512 elements, matching 512 bytes. The current lecture preserves this callback signature and cast.

03 | Complete the memory append – Problem

GUIDED CODE Raw buffer

Fill capacity, copy, and terminator

```
int write_data2(void* ptr, size_t dataSize,
                size_t nmemb, void* data)
{
    size_t realsize = dataSize * nmemb;
    MemoryStruct* mem = (MemoryStruct*)data;
    mem->memory = (char*)myrealloc(
        mem->memory, /* new size */);
    if (mem->memory) {
        /* copy new bytes at old end */
        mem->size += realsize;
        /* add null terminator */
    }
    return (int)realsize;
}
```

03 | Complete the memory append – Solution

```
mem->memory = (char*)myrealloc(
    mem->memory, mem->size + realsize + 1);

if (mem->memory) {
    memcpy(&(mem->memory[mem->size]),
           ptr, realsize);
    mem->size += realsize;
    mem->memory[mem->size] = 0;
}
return (int)realsize;
```

Why add one?

The response bytes occupy indices 0 through `size-1`; index `size` stores `'\0'` so the buffer can be read as a C string.

04 | Trace the buffer size – Problem

GUIDED CODE Memory invariant Compute before revealing

The callback starts with `size = 0`. Libcurl delivers blocks of 120, 80, and 50 bytes.

1. What are the successive values of `size`?
2. How many bytes must be allocated after the final block?
3. At which index is the null terminator stored?

04 | Trace the buffer size – Problem

GUIDED CODE Memory invariant Compute before revealing

The callback starts with `size = 0`. Libcurl delivers blocks of 120, 80, and 50 bytes.

1. What are the successive values of `size`?
2. How many bytes must be allocated after the final block?
3. At which index is the null terminator stored?

Answer

The sizes are 120, 200, and 250. The final allocation needs 251 bytes, and `memory[250]` stores the terminator.

The class stores configuration and one handle

```
class EODMarketData {
private:
    std::vector<std::string> symbol_list;
    std::string symbol_file, api_token_file;
    std::string result_file, url_common;
    std::string start_date, end_date, api_token;
    CURL* handle;

    void load_symbols();
    std::string read_api_token(
        const std::string& filename) const;
    std::string build_request_url(
        const std::string& symbol) const;
    void set_common_curl_options(
        const std::string& requestUrl);
};
```

Responsibility map

Function	Responsibility
`load_symbols`	read the first CSV field into `symbol_list`
`read_api_token`	load credentials from a separate file
`build_request_url`	combine symbol, dates, token, and period
`set_common_curl_options`	configure URL, user agent, and TLS checks
`fetch_data_to_*`	choose a destination and perform one request
`parse_data`	extract date and adjusted close from raw CSV

Design benefit

Each function has one reason to fail, which makes error messages and tests easier to interpret.

05 | Complete construction – Problem

GUIDED CODE Resource acquisition

Order the setup steps

```
EODMarketData::EODMarketData(/* inputs */)
  : symbol_file(symbolFile_),
    api_token_file(apiTokenFile_),
    result_file(resultFile_),
    url_common("https://eodhistoricaldata.com/api/eod/"),
    start_date(startDate_), end_date(endDate_),
    handle(NULL)
{
    /* load tickers */
    /* read token */
    /* global curl initialization */
    /* create easy handle and validate it */
}
```

05 | Complete construction – Solution

```
load_symbols();
api_token = read_api_token(api_token_file);
curl_global_init(CURL_GLOBAL_ALL);
handle = curl_easy_init();

if (!handle) {
    curl_global_cleanup();
    throw runtime_error("Curl init failed!");
}
```

Constructor guarantee

If construction succeeds, the object has symbols, a token, and a usable handle. If handle creation fails, global curl state is cleaned before the exception leaves.

The destructor closes the lifecycle

```
EODMarketData::~EODMarketData()
{
    if (handle) {
        curl_easy_cleanup(handle);
        handle = NULL;
    }
    curl_global_cleanup();
}
```

RAII connection

The constructor acquires the resource and the destructor releases it. Stack unwinding invokes the destructor when later code throws an exception.

06 | Read the API token – Problem

GUIDED CODE File input

Complete success and failure paths

```
string EODMarketData::read_api_token(
    const string& filename) const
{
    ifstream fin(filename);
    string token;
    if (/* file opened */) {
        /* read one line */;
    }
    else {
        /* report filename in an exception */;
    }
    return token;
}
```

06 | Read the API token – Solution

```
ifstream fin(filename);
string token;
if (fin.is_open()) {
    getline(fin, token);
    fin.close();
}
else {
    throw runtime_error(
        "Failed to open API token file: " + filename);
}
return token;
```

Credential boundary

The token lives in a separate file. The program reads it but should not print it or commit that file to the repository.

The supplied symbol file is a CSV table

```
Ticker,Name,Sector,Asset Class,Market Value,...
NVDA,NVIDIA CORP,Information Technology,Equity,...
AAPL,APPLE INC,Information Technology,Equity,...
MSFT,MICROSOFT CORP,Information Technology,Equity,...
```

What the downloader needs

Only the first field, `Ticker`, is required to build market-data requests. The remaining fields describe the ETF holdings.

GUIDED CODE CSV input

Skip the header and extract field zero

```
void EODMarketData::load_symbols()
{
    ifstream fin(symbol_file);
    if (!fin.is_open())
        throw runtime_error(/* message */);

    string line, symbol;
    /* discard header */;
    while (getline(fin, line)) {
        stringstream sin(line);
        /* read until first comma */;
        if (!symbol.empty())
            symbol_list.push_back(symbol);
    }
}
```

07 | Load the symbols – Solution

```
if (!fin.is_open())
    throw runtime_error(
        "Failed to open symbol file: " + symbol_file);

string line, symbol;
getline(fin, line);              // skip header
while (getline(fin, line)) {
    stringstream sin(line);
    getline(sin, symbol, ',');   // first field
    if (!symbol.empty())
        symbol_list.push_back(symbol);
}
```

Why this limited parser is sufficient here

The required ticker is the first field and contains no comma. This is not a general-purpose CSV parser for arbitrary quoted fields.

Dataset diagnostic before the network

The supplied holdings file contains:

- 2,580 data rows;
- 2,580 distinct ticker strings;
- a header whose first field is `Ticker`;
- leading examples `NVDA`, `AAPL`, `MSFT`, `AMZN`, `GOOGL`.

Question

If `symbol_list.size()` is 2,581 after loading, what is the most likely mistake?

Dataset diagnostic before the network

The supplied holdings file contains:

- 2,580 data rows;
- 2,580 distinct ticker strings;
- a header whose first field is `Ticker`;
- leading examples `NVDA`, `AAPL`, `MSFT`, `AMZN`, `GOOGL`.

Question

If `symbol_list.size()` is 2,581 after loading, what is the most likely mistake?

The header was probably not skipped.

08 | Build the request URL – Problem

GUIDED CODE String composition

Complete the query

```
string EODMarketData::build_request_url(
    const string& symbol) const
{
    return url_common + symbol + ".US"
        + "?from=" + /* ? */
        + "&to=" + /* ? */
        + "&api_token=" + /* ? */
        + "&period=d";
}
```

For `NVDA`, which part is the path and which parts are query parameters?

08 | Build the request URL – Solution

```
return url_common + symbol + ".US"
    + "?from=" + start_date
    + "&to=" + end_date
    + "&api_token=" + api_token
    + "&period=d";
```

Result shape

`/api/eod/NVDA.US` selects the endpoint and instrument. The query string supplies the date interval, credential, and daily period.

Debug safely

Inspect the symbol and dates separately. Avoid logging the complete URL because it contains the API token.

Configure the options shared by both destinations

```
void EODMarketData::set_common_curl_options(
    const string& requestUrl)
{
    curl_easy_setopt(
        handle, CURLOPT_URL, requestUrl.c_str());
    curl_easy_setopt(handle, CURLOPT_USERAGENT,
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:74.0) "
        "Gecko/20100101 Firefox/74.0");
    curl_easy_setopt(
        handle, CURLOPT_SSL_VERIFYPEER, 1L);
    curl_easy_setopt(
        handle, CURLOPT_SSL_VERIFYHOST, 2L);
}
```

GUIDED CODE Transport checks — Match option to meaning

Setting	Meaning
`CURLOPT_SSL_VERIFYPEER = 1L`	verify the certificate chain
`CURLOPT_SSL_VERIFYHOST = 2L`	verify certificate name against host

Course-code improvement

The 2026 material enables both checks. Disabling them may hide a local setup problem and weakens the identity guarantee of the HTTPS connection.

10 | Fetch to a file – Problem

GUIDED CODE Learning path

Complete callback and destination

```
FILE* fp = fopen(result_file.c_str(), "ab");
if (!fp)
    throw runtime_error("Failed to open results file");

set_common_curl_options(build_request_url(symbol));
fprintf(fp, "%s\n", symbol.c_str());
curl_easy_setopt(handle, CURLOPT_WRITEFUNCTION,
                 /* callback */);
curl_easy_setopt(handle, CURLOPT_WRITEDATA,
                 /* destination */);
CURLcode status = curl_easy_perform(handle);
```

10 | Fetch to a file – Solution

```
curl_easy_setopt(handle, CURLOPT_WRITEFUNCTION,
                 write_data);
curl_easy_setopt(handle, CURLOPT_WRITEDATA, fp);

CURLcode status = curl_easy_perform(handle);
fprintf(fp, "%c", '\n');
fclose(fp);

if (status != CURLE_OK)
    throw runtime_error(
        string("curl_easy_perform() failed: ")
        + curl_easy_strerror(status));
```

Why both "wb" and "ab"?

Before the loop

Open `Results.txt` once with `"wb"` and close it immediately. This truncates any output from a previous run.

Inside the loop

Open with `"ab"` for each symbol. This appends that symbol and response without deleting earlier results from the current run.

Prediction

What would happen if every per-symbol call used `"wb"`?

11 | Fetch to memory – Problem

GUIDED CODE Efficient path

Complete destination, validation, and cleanup

```
MemoryStruct data;
set_common_curl_options(build_request_url(symbol));
curl_easy_setopt(handle, CURLOPT_WRITEFUNCTION,
                 /* callback */);
curl_easy_setopt(handle, CURLOPT_WRITEDATA,
                 /* destination */);

CURLcode status = curl_easy_perform(handle);
if (status != CURLE_OK) {
    /* release buffer, then throw */
}
/* parse response, then release buffer */
```

11 | Fetch to memory – Solution

```
curl_easy_setopt(handle, CURLOPT_WRITEFUNCTION,
                 write_data2);
curl_easy_setopt(handle, CURLOPT_WRITEDATA,
                 (void*)&data);
CURLcode status = curl_easy_perform(handle);
if (status != CURLE_OK) {
    free(data.memory);
    throw runtime_error(
        string("curl_easy_perform() failed: ")
        + curl_easy_strerror(status));
}
parse_data(symbol, data.memory);
free(data.memory);
```

Both paths release memory

The buffer is freed after a successful parse and before the exception on transfer failure.

Know the response before parsing it

A daily EOD response is expected to resemble:

```
date,open,high,low,close,adjusted_close,volume
2026-05-28,134.10,137.90,133.70,137.35,137.35,20123456
2026-05-29,137.20,138.10,135.90,136.75,136.75,18234567
```

Target fields

Topic 11 prints the first field as the date and the final price field as the adjusted close, ignoring the remaining values.

12 | Parse date and adjusted close – Problem

GUIDED CODE String parsing

Complete three transformations

```
while (getline(sData, line)) {
    if (line.empty()) continue;
    if (!(line[0] >= '0' && line[0] <= '9'))
        continue;

    sDate = /* text before first comma */;
    line.erase(/* remove final field */);
    sValue = /* text after new final comma */;
    dValue = strtod(sValue.c_str(), NULL);
}
```

Why does the first-character check skip both the CSV header and textual API errors?

12 | Parse date and adjusted close – Solution

```
sDate = line.substr(0, line.find_first_of(','));

line.erase(line.find_last_of(','));
sValue = line.substr(line.find_last_of(',') + 1);
dValue = strtod(sValue.c_str(), NULL);

cout << sDate << " " << fixed
     << setprecision(2) << dValue << endl;
```

Two-step tail extraction

The first `find_last_of` removes volume. After that erasure, the new final field is adjusted close.

13 | Which inputs should be skipped?

GUIDED CODE Parser guard

Classify each line

1. `date,open,high,low,close,adjusted_close,volume`
2. `2026-05-29,137.20,138.10,135.90,136.75,136.75,18234567`
3. `Invalid API key`
4. an empty line

13 | Which inputs should be skipped?

GUIDED CODE Parser guard — Classify each line

1. `date,open,high,low,close,adjusted_close,volume`
2. `2026-05-29,137.20,138.10,135.90,136.75,136.75,18234567`
3. `Invalid API key`
4. an empty line

Answer

Only line 2 begins with a date digit and reaches the numeric extraction. The header, textual error, and empty line are skipped.

Parsing still needs assumptions

The lecture parser assumes every accepted line:

- starts with an ISO-style date;
- contains enough comma-separated fields;
- places volume last and adjusted close immediately before it;
- contains a valid numeric adjusted close.

Diagnostic rule

When parsing fails, print or save a short redacted response sample first. A server error page can be syntactically different from the expected CSV.

Network and parser errors are different

Transfer failure

`curl_easy_perform` returns a non-success `CURLcode`: DNS, TLS, timeout, or connection failure.

Content failure

The transfer succeeds, but the response contains an API error, unexpected schema, or malformed data.

Question

Why can checking only `status == CURLE_OK` still produce no parsed prices?

Two run modes reuse the same symbol list

```
void EODMarketData::run_for_learning()
{
    FILE* fp = fopen(result_file.c_str(), "wb");
    if (!fp)
        throw runtime_error(
            "Failed to initialize results file");
    fclose(fp);

    for (const auto& symbol : symbol_list)
        fetch_data_to_file(symbol);
}

void EODMarketData::run_for_efficency()
{
    for (const auto& symbol : symbol_list)
        fetch_data_to_memory(symbol);
}
```

File path or memory path?

	File path	Memory path
Best for	learning and inspection	immediate processing
Intermediate artifact	`Results.txt`	`MemoryStruct`
Callback	`write_data`	`write_data2`
Extra work	open/append/close	allocate/copy/free
Parser timing	later	immediately after transfer

Same transfer layer

Both paths use the same handle, URL builder, user agent, TLS settings, and `curl_easy_perform` call.

GUIDED CODE Exception handling

Predict each exit path

```cpp
int main()
{
    try {
        EODMarketData data(symbolFile, "api_token.txt",
                           "Results.txt",
                           "2025-01-01", "2026-05-31");
        data.run_for_efficency();
    }
    catch (const exception& ex) {
        cerr << "Exception: " << ex.what() << endl;
        return -1;
    }
    catch (...) {
        cerr << "Unknown exception occurred" << endl;
        return -1;
    }
    return 0;
}
```

Diagnose by pipeline stage

Symptom	Likely stage	First check
No tickers	local input	file path, header skip
`Curl init failed`	handle creation	global init and environment
Transfer error	network/TLS	`curl_easy_strerror(status)`
Transfer succeeds, no rows	response/parser	redacted response prefix
Only last symbol in file	output mode	accidental `"wb"` per symbol
Memory grows indefinitely	ownership	missing `free` path

15 | Test parsing without the network – Problem

GUIDED CODE Deterministic test — Supply a local response string

```
const char* rawData =
    "date,open,high,low,close,adjusted_close,volume\n"
    "2026-05-29,137.20,138.10,135.90,"
    "136.75,136.75,18234567\n";

parse_data("NVDA", rawData);
```

What exact date and value should be printed? Which parts of the full pipeline are deliberately excluded from this test?

15 | Test parsing without the network – Solution

Expected parsed output:

```
2026-05-29 136.75
```

What the test isolates

It verifies line filtering, date extraction, removal of volume, adjusted-close extraction, numeric conversion, and formatting.

What it does not test

DNS, TLS, authentication, the remote schema, libcurl callbacks, or network availability.

Ownership checklist

- `EODMarketData` owns the easy handle.
- The constructor creates it; the destructor cleans it.
- `fetch_data_to_file` owns its `FILE*` until `fclose`.
- `fetch_data_to_memory` owns `data.memory` until `free`.
- The callbacks borrow their destination pointers only during the call.
- `parse_data` reads the raw buffer but does not free it.

Debugging question

For every pointer, ask: who creates it, who may use it, and who releases it on both success and failure?

Exit ticket

Explain

Why does libcurl need a write callback plus a destination pointer?

Diagnose

The transfer returns `CURLE_OK`, but the parser prints nothing. Name two checks.

Predict

What remains in `Results.txt` if every ticker request opens it with `"wb"`?

Connect

Which historical-price output becomes input for Topic 12 visualization?

Reserve exercises

Optional challenges and operational checks

R1 | Find the allocation failure bug

GUIDED CODE Challenge

What pointer can be lost?

```
mem->memory = (char*)realloc(
    mem->memory, mem->size + realsize + 1);
if (mem->memory) {
    memcpy(mem->memory + mem->size, ptr, realsize);
}
```

If `realloc` fails, what happens to the address of the old allocation?

R1 | Find the allocation failure bug

GUIDED CODE Challenge

What pointer can be lost?

```
mem->memory = (char*)realloc(
    mem->memory, mem->size + realsize + 1);
if (mem->memory) {
    memcpy(mem->memory + mem->size, ptr, realsize);
}
```

If `realloc` fails, what happens to the address of the old allocation?

Answer

The direct assignment overwrites the only stored pointer with `NULL`; the old allocation still exists but can no longer be freed. A temporary pointer should be checked before updating `mem->memory`.

R1 | Preserve the old pointer – Solution

```
char* resized = (char*)realloc(
    mem->memory, mem->size + realsize + 1);
if (!resized)
    return 0;

mem->memory = resized;
memcpy(mem->memory + mem->size, ptr, realsize);
mem->size += realsize;
mem->memory[mem->size] = 0;
return realsize;
```

Callback signal

Returning fewer bytes than libcurl supplied tells libcurl that the write failed.

R2 | Optional modern buffer callback

GUIDED CODE Challenge — Use a standard-library owner

```
size_t write_to_string(void* ptr, size_t size,
                       size_t nmemb, void* data)
{
    size_t bytes = size * nmemb;
    std::string* output =
        static_cast<std::string*>(data);
    output->append(static_cast<char*>(ptr), bytes);
    return bytes;
}
```

Why this is optional

It removes manual `malloc/realloc/free` ownership, but the main deck follows the exact Topic 11 `MemoryStruct` implementation.

GUIDED CODE Challenge — Add a response-code check

```
long httpCode = 0;
curl_easy_getinfo(handle,
                  CURLINFO_RESPONSE_CODE,
                  &httpCode);
```

Question

Why might `curl_easy_perform` return `CURLE_OK` while `httpCode` is 401, 403, or 500?

GUIDED CODE Challenge — Add a response-code check

```
long httpCode = 0;
curl_easy_getinfo(handle,
                  CURLINFO_RESPONSE_CODE,
                  &httpCode);
```

Question

Why might `curl_easy_perform` return `CURLE_OK` while `httpCode` is 401, 403, or 500?

The network transfer completed successfully, but the server returned an application-level error response.

R4 | Keep credentials out of diagnostics

Suppose a failing request URL contains

```
&api_token=SECRET&period=d.
```

Problem

Design a diagnostic message that identifies symbol, dates, and error status without exposing the token.

R4 | Keep credentials out of diagnostics

Suppose a failing request URL contains

```
&api_token=SECRET&period=d.
```

Problem

Design a diagnostic message that identifies symbol, dates, and error status without exposing the token.

Example:

```
NVDA 2025-01-01..2026-05-31:  HTTP 401
```

Rule

Treat complete authenticated URLs as credentials, not ordinary log strings.

R5 | Validate an empty token file

GUIDED CODE Challenge

Add a content check

```
getline(fin, token);
if (token.empty())
    throw runtime_error(
        "API token file is empty: " + filename);
```

Follow-up

What additional issue can arise if the token file was created with a different line-ending convention? Inspect the final character before building the URL.

R6 | Validate the date interval

ISO dates in `YYYY-MM-DD` format compare in chronological order when they are valid and zero-padded.

Problem

Which checks should run before any network request?

- both strings contain ten characters;
- hyphens appear at positions 4 and 7;
- remaining characters are digits;
- `start_date <= end_date`.

Full calendar validation is a separate challenge.

R7 | EC2 build-and-run checklist

```
sudo apt-get update
sudo apt install libcurl4-openssl-dev -y

make clean
make
./EODMarketData
```

Before blaming the code

Confirm the symbol CSV and `api_token.txt` are in the working directory expected by the executable, then inspect the first requested symbol and first response.

Market-data pipeline – One-page cheat sheet

Transfer

Initialize handle → set URL and TLS options → set callback and destination → perform → check status.

Parse

Validate response shape, skip non-data lines, extract date and adjusted close, then convert and format.

Local inputs

Skip the symbol-file header, read ticker field zero, and keep the API token in a separate untracked file.

Ownership

Close every file, free every response buffer, and clean the easy handle through the object's destructor.